

Final Exam

- Time: 10:30-12:30
- Do not use “písané písmo”; write in “paličkové písmo”.
- Toilet breaks are not allowed during the exam.
- Write your answers on the provided answer sheets. Additional sheets will be supplied upon request. Please ensure that your full name is clearly written on every page.
- Include detailed computation steps for all solutions. Answers without supporting calculations will receive no credit.

Question 1. (5 marks)

1. (2.5 marks) Find $\gcd(1105, 68)$ using the Euclidean algorithm.
2. (2.5 marks) Find $17^{-1} \bmod 160$ using the extended Euclidean algorithm.

Solution. 1. By the Euclidean algorithm

$$\begin{aligned} 1105 &= 68 \times 16 + 17 & \gcd(1105, 68) &= \gcd(68, 17) & (1 \text{ mark}) \\ 68 &= 17 \times 4 & \gcd(68, 17) &= 17 & (1 \text{ mark}) \end{aligned}$$

We have $\gcd(1105, 68) = 17$. (0.5 marks)

2. By the Euclidean algorithm:

$$\begin{aligned} 160 &= 17 \times 9 + 7 & 17 &= 7 \times 2 + 3 \\ 7 &= 3 \times 2 + 1 & & (1 \text{ mark}) \end{aligned}$$

By the extended Euclidean algorithm:

$$\begin{aligned} 1 &= 7 - 3 \times 2, & 3 &= 17 - 7 \times 2 \\ 7 &= 160 - 17 \times 9 \end{aligned}$$

$$\begin{aligned} 1 &= 7 - 3 \times 2 = 7 - (17 - 7 \times 2) \times 2 = 7 \times 5 - 17 \times 2 \\ &= (160 - 17 \times 9) \times 5 - 17 \times 2 = 160 \times 5 - 17 \times 47 & (1 \text{ mark}) \end{aligned}$$

Thus $17^{-1} \bmod 160 = -47 \bmod 160 = 113$. (0.5 marks)

Question 2. (5 marks) Solve the following system of linear congruences:

$$\begin{aligned} x &\equiv 2 \bmod 3 \\ x &\equiv 4 \bmod 5 \\ x &\equiv 6 \bmod 7 \\ x &\equiv ? \bmod 105 \end{aligned}$$

Solution. Using the formula from the lecture,

$$m_1 = 3, \quad m_2 = 5, \quad m_3 = 7, \quad a_1 = 2, \quad a_2 = 4, \quad a_3 = 6,$$

$$m = 3 \times 5 \times 7 = 105, \quad M_1 = 35, \quad M_2 = 21, \quad M_3 = 15. \quad (1.5 \text{ marks})$$

Then

$$M_1 \equiv 35 \equiv 2 \pmod{3}, \quad M_2 \equiv 21 \equiv 1 \pmod{5}, \quad M_3 \equiv 15 \equiv 1 \pmod{7}.$$

Using the extended Euclidean algorithm, we find

$$y_1 = M_1^{-1} \pmod{3} = 2, \quad y_2 = M_2^{-1} \pmod{5} = 1, \quad y_3 = M_3^{-1} \pmod{7} = 1. \quad (1.5 \text{ marks})$$

Therefore,

$$\begin{aligned} x &= \sum_{i=1}^3 a_i y_i M_i \pmod{m} = 2 \times 2 \times 35 + 4 \times 1 \times 21 + 6 \times 1 \times 15 \pmod{105} \\ &= 314 \pmod{105} = 104 \pmod{105}. \quad (2 \text{ marks}) \end{aligned}$$

Question 3. (10 marks) We have introduced a method for implementing masked PRESENT by using precomputed lookup tables, denoted as T1 and T2. For any $\mathbf{v} \in \mathbb{F}_2^4$, any input mask $\mathbf{m}_{\text{in}} \in \mathbb{F}_2^4$, and the corresponding output mask $\mathbf{m}_{\text{out}} \in \mathbb{F}_2^4$ for the PRESENT Sbox, the table T1 is defined as follows:

$$\text{T1}[\mathbf{v} \oplus \mathbf{m}_{\text{in}}, \mathbf{m}_{\text{in}}] = \text{SB}(\mathbf{v}) \oplus \mathbf{m}_{\text{out}}. \quad (1)$$

Additionally, T2 is used to keep track of the mask transitions:

$$\text{T2}[\mathbf{m}_{\text{in}}] = \mathbf{m}_{\text{out}}, \quad \mathbf{m}_{\text{in}} = 0, 1, \dots, \mathbf{F}. \quad (2)$$

This approach eliminates the need to regenerate a masked Sbox table each time the input mask changes.

Please complete the masked PRESENT algorithm by filling in each missing line based on the specific instructions provided.

- (2 marks) For line 7, implement the computation of the masked Sbox.
- (2 marks) For line 8, store the resulting output masks from the Sbox operation.
- (2 marks) For line 9, apply the pLayer operation to the current cipher state.
- (2 marks) For line 10, apply the pLayer operation to the masks.
- (2 marks) For line 12, remove masks from the ciphertext.

Algorithm 1: Masked implementation of PRESENT.

Input: p , T1, T2, K_i ($i = 1, 2, \dots, 32$)// p is the plaintext for encryption; T1 is the table for masked Sbox as given in Equation 1; T2 specifies the output mask given the input mask for PRESENT Sbox as defined in Equation 2; K_i are round keys for PRESENT encryption

Output: Ciphertext

```

1 randomly generate 16 masks     $m_0, m_1, \dots, m_{15}$ 
2 array of size 16    state =  $p \oplus m_{15}, m_{14}, \dots, m_1, m_0$ // mask the  $j$ th nibble of the
    plaintext with  $m_j$ ; each entry of the array is one masked nibble
3 array of size 16    masks =  $m_{15}, m_{14}, \dots, m_1, m_0$ 
4 for  $i = 1, i \leq 31, i++$  do
5     state = addRoundKey(state,  $K_i$ )
6     for  $j = 0, j < 16, j++$  do
7         // for each nibble
8         ?// masked Sbox computation
9         ?// record the output masks of Sbox computation
10    ?// apply pLayer to the cipher state
11    ?// apply pLayer to the masks
11 state = addRoundKey(state,  $K_{32}$ )
12 ?// remove masks from the ciphertext
13 return state

```

Algorithm 2: Masked implementation of PRESENT.

Input: p , T1, T2, K_i ($i = 1, 2, \dots, 32$)// p is the plaintext for encryption; T1 is the table for masked Sbox as given in Equation 1; T2 specifies the output mask given the input mask for PRESENT Sbox as defined in Equation 2; K_i are round keys for PRESENT encryption

Output: Ciphertext

```

1 randomly generate 16 masks     $m_0, m_1, \dots, m_{15}$ 
2 array of size 16    state =  $p \oplus m_{15}, m_{14}, \dots, m_1, m_0$ // mask the  $j$ th nibble of the
    plaintext with  $m_j$ ; each entry of the array is one masked nibble
3 array of size 16    masks =  $m_{15}, m_{14}, \dots, m_1, m_0$ 
4 for  $i = 1, i \leq 31, i++$  do
5     state = addRoundKey(state,  $K_i$ )
6     for  $j = 0, j < 16, j++$  do
7         // for each nibble
8         state[j] = T1[state[j], masks[j]]// masked Sbox computation
9         masks[j] = T2[masks[j]]// record the output masks of Sbox computation
10    state = pLayer(state)// apply pLayer to the cipher state
11    masks = pLayer(masks)// apply pLayer to the masks
11 state = addRoundKey(state,  $K_{32}$ )
12 state = state  $\oplus$  masks
13 return state

```

Solution.

Question 4. (10 marks) In this question, we consider RSA encryption and decryption. Suppose Bob wants to generate his RSA private and public keys. Bob randomly generates $p = 29$ and $q = 41$.

- a) (1 mark) Compute

$$n = ? \quad \varphi(n) = ?$$

- b) (2 marks) From $\mathbb{Z}_{\varphi(n)}^*$, Bob chooses his public key exponent $e = 3$. Use the extended Euclidean algorithm to compute Bob's corresponding private key d .

- c) (1 mark) Alice wants to send the plaintext $m = 2$ to Bob using Bob's public key n, e . Alice computes ciphertext

$$c = ?$$

- d) (3 marks) Bob receives a message $c = 155$ from Alice. To decrypt the message $c = 155$ using CRT-based RSA, Bob computes

$$m_p = ? \quad m_q = ? \quad y_p = ? \quad y_q = ?$$

Using Garner's algorithm, Bob gets

$$m = ?$$

- e) (3 marks) Now we consider a malicious attacker who carries out a Bellcore attack during the RSA decryption. Suppose she injects a fault during the computation of m_p , resulting in the faulty value $m'_p = 1$. Then the faulty message will be

$$m' = ?$$

Given m , m' , and the public key n, e , how can the attacker recover the secret key d ?

Solution.

- a) (1 mark)

$$n = 29 \times 41 = 1189, \quad \varphi(n) = 28 \times 40 = 1120.$$

- b) (2 marks) By the extended Euclidean algorithm

$$1120 = 3 \times 373 + 1 \implies 1 = 1120 - 3 \times 373. \quad (1 \text{ mark})$$

Hence Bob's private key $d = -373 \bmod 1120 = 747$. (1 mark)

- c) (1 mark) Alice computes

$$c = m^e \bmod n = 2^3 \bmod 1189 = 8.$$

- d) (3 marks) After receiving the ciphertext $c = 155$, with CRT-based RSA implementation, Bob computes

$$\begin{aligned} m_p &= c^{d \bmod (p-1)} \bmod p = 155^{747 \bmod 28} \bmod 29 = 10^{19} \bmod 29 = 21, \\ m_q &= c^{d \bmod (q-1)} \bmod q = 155^{747 \bmod 40} \bmod 41 = 32^{27} \bmod 41 = 9. \quad (1 \text{ mark}) \end{aligned}$$

To compute $10^{19} \bmod 29$, we note that

$$\begin{aligned} 10^2 \bmod 29 &= 100 \bmod 29 = 13, \\ 10^4 \bmod 29 &= 13^2 \bmod 29 = 24, \\ 10^8 \bmod 29 &= 24^2 \bmod 29 = 25, \\ 10^{16} \bmod 29 &= 25^2 \bmod 29 = 16. \end{aligned}$$

Thus

$$10^{19} \bmod 29 = 10^{16} \times 10^2 \times 10 \bmod 29 = 16 \times 13 \times 10 \bmod 29 = 21.$$

Similarly,

$$\begin{aligned} 32^2 \bmod 41 &= 40 \\ 32^3 \bmod 41 &= 32 \times 40 \bmod 41 = 9 \\ 32^9 \bmod 41 &= 9^3 \bmod 41 = 32 \\ 32^{27} \bmod 41 &= 32^3 \bmod 41 = 9. \end{aligned}$$

By the extended Euclidean algorithm

$$41 = 29 + 12, \quad 29 = 12 \times 2 + 5, \quad 12 = 5 \times 2 + 2, \quad 5 = 2 \times 2 + 1,$$

which gives

$$\begin{aligned} 1 &= 5 - 2 \times (12 - 5 \times 2) = -2 \times 12 + (29 - 12 \times 2) \times 5 \\ &= 29 \times 5 - 12 \times (41 - 29) = -41 \times 12 + 29 \times 17. \end{aligned}$$

We have

$$\begin{aligned} y_p &= p^{-1} \bmod q = 29^{-1} \bmod 41 = 17 \bmod 41, \\ y_q &= q^{-1} \bmod p = 41^{-1} \bmod 29 = -12 \bmod 29 = 17 \bmod 29. \quad (1 \text{ mark}) \end{aligned}$$

By Garner's algorithm,

$$\begin{aligned} m &= m_p + ((m_q - m_p)y_p \bmod q)p = 21 + ((9 - 21) \times 17 \bmod 41) \times 29 \\ &= 21 + 1 \times 29 = 50. \quad (1 \text{ mark}) \end{aligned}$$

e) (3 marks) The faulty message is given by

$$\begin{aligned} m' &= m'_p + ((m'_q - m'_p)y_p \bmod q)p = 1 + ((9 - 1) \times 17 \bmod 41) \times 29 \\ &= 1 + 13 \times 29 = 378. \quad (1 \text{ mark}) \end{aligned}$$

Using the Bellcore attack,

$$q = \gcd(m' - m, n) = \gcd(378 - 50, 1189) = \gcd(328, 1189).$$

By the Euclidean algorithm

$$\begin{aligned} 1189 &= 328 \times 3 + 205, & \gcd(328, 1189) &= \gcd(328, 205), \\ 328 &= 205 \times 1 + 123, & \gcd(328, 205) &= \gcd(205, 123) \\ 205 &= 123 \times 1 + 82, & \gcd(205, 123) &= \gcd(123, 82) \\ 123 &= 82 \times 1 + 41, & \gcd(123, 82) &= \gcd(82, 41) = 41. \quad (1 \text{ mark}) \end{aligned}$$

Hence $q = 41$ and

$$p = \frac{n}{q} = 29.$$

The attacker can then compute

$$\varphi(n) = (p - 1)(q - 1) = 1120.$$

Using the extended Euclidean algorithm again, they obtain

$$d = e^{-1} \bmod \varphi(n) = 3^{-1} \bmod 1120 = 747. \quad (1 \text{ mark})$$

Thus the attacker recovers the secret key $d = 747$.

Question 5. (10 marks)

Definition 1. For an Sbox $SB : \mathbb{F}_2^{\omega_1} \rightarrow \mathbb{F}_2^{\omega_2}$, the (*extended*) *difference distribution table (DDT)* of SB is a 2-dimensional table T of size $2^{\omega_2} \times (2^{\omega_1} - 1)$ such that for any $0 < \delta < 2^{\omega_1}$ and $0 \leq \Delta < 2^{\omega_2}$, the entry of T at the Δ th row and δ th column is given by

$$T[\Delta, \delta] = \{ \mathbf{a} \mid \mathbf{a} \in \mathbb{F}_2^{\omega_1}, SB(\mathbf{a} \oplus \delta) \oplus SB(\mathbf{a}) = \Delta \}.$$

Here we identify elements of $\mathbb{F}_2^{\omega_1}$ and $\mathbb{F}_2^{\omega_2}$ with their binary representations as integers when indexing the rows and columns of the table. We refer to δ as the *input difference*, and Δ as the *output difference*.

Let SB be the following Sbox

x	0	1	2	3	4	5	6	7
$SB(x)$	4	7	0	5	2	6	3	1

Define

$$\begin{aligned} f : \mathbb{F}_2^3 &\rightarrow \mathbb{F}_2^3 \\ \mathbf{x} &\mapsto SB(\mathbf{x} \oplus \mathbf{a}) \oplus \mathbf{b}. \end{aligned}$$

We consider a differential fault analysis attack on f . Our attack assumption is as follows:

- Fault location: input of f
- Fault model: bit flip
- Fault mask: $\varepsilon \in \mathbb{F}_2^3$ such that $\mathbf{x}' = \mathbf{x} \oplus \varepsilon$
- Attacker knowledge: Sbox design, inputs and outputs of f , fault mask
- Attacker goal: recover values of $\mathbf{a}$ and $\mathbf{b}$
- The attacker can repeat the computation with the same input, which is not chosen by the attacker.

We know that with input $\mathbf{x} = 3$, the correct output is 4.

- a) (4 marks) Complete the DDT for SB. (In this table, the empty row corresponding to $\Delta = 0$ is omitted.)

$\Delta \backslash \delta$	1	2	3	4	5	6	7
1	?	?	?	?			
2	?	13			05	24	
3	01		?	?		?	
4	45	02		37			16
5	23		56		14		07
6				04	36	17	25
7		57	12			06	34

- b) (2 marks) Suppose the attacker injects a fault with fault mask $\varepsilon_1 = 2$ and the resulting faulty output is 6. What can we conclude about $\mathbf{a}$?

- c) (4 marks) Suppose the attacker injects another fault with fault mask $\varepsilon_2 = 3$ and the resulting faulty output is 3. Determine the values of $\mathbf{a}$ and $\mathbf{b}$.

Solution.

- a) (4 marks)

$\Delta \backslash \delta$	1	2	3	4	5	6	7
1		46	03	15	27		
2	67	13			05	24	
3	01		47	26		35	
4	45	02		37			16
5	23		56		14		07
6				04	36	17	25
7		57	12			06	34

0.5 marks for each missing entry

- b) (2 marks) For a fault mask ε , let Δ denote the difference between the correct and faulty outputs. Then

$$\begin{aligned}\Delta &= (\text{SB}(\mathbf{x} \oplus \mathbf{a}) \oplus \mathbf{b}) \oplus (\text{SB}(\mathbf{x}' \oplus \mathbf{a}) \oplus \mathbf{b}) = \text{SB}(\mathbf{x} \oplus \mathbf{a}) \oplus \text{SB}(\mathbf{x}' \oplus \mathbf{a}) \\ &= \text{SB}(\mathbf{x} \oplus \mathbf{a}) \oplus \text{SB}(\mathbf{x} \oplus \mathbf{a} \oplus \varepsilon).\end{aligned}$$

We can conclude that the value $\mathbf{x} \oplus \mathbf{a}$ belongs to the DDT entry corresponding to input difference $\delta = \varepsilon$ and output difference Δ .

With fault mask $\varepsilon_1 = 2$ and

$$\Delta = 4 \oplus 6 = 2, \quad (1 \text{ mark})$$

we know that the value $\mathbf{x} \oplus \mathbf{a}$ is in the entry of DDT corresponding to input difference 2 and output difference 2. Thus the possible values of $\mathbf{x} \oplus \mathbf{a}$ are 1 and 3. Knowing that $\mathbf{x} = 3$, the possible values of $\mathbf{a}$ are 2 and 0. (1 mark)

- c) (4 marks) Similarly, with fault mask $\varepsilon_2 = 3$ and

$$\Delta = 4 \oplus 3 = 7, \quad (1 \text{ mark})$$

we know that the value $\mathbf{x} \oplus \mathbf{a}$ is in the entry of DDT corresponding to input difference 3 and output difference 7. Thus the possible values of $\mathbf{x} \oplus \mathbf{a}$ are 1 and 2. (1 mark)

Knowing that $\mathbf{x} = 3$, the possible values of $\mathbf{a}$ are 2 and 1. Together with the previous answer, this implies that $\mathbf{a} = 2$. (1 mark)

We also know that when $\mathbf{x} = 3$,

$$\text{SB}(3 \oplus \mathbf{a}) \oplus \mathbf{b} = 4,$$

which gives

$$\mathbf{b} = \text{SB}(3 \oplus 2) \oplus 4 = \text{SB}(1) \oplus 4 = 7 \oplus 4 = 3. \quad (1 \text{ mark})$$